

DD: SUB-WF-RELOCATION-01-FLOW-WIK — KE WIK

Relocation BPMN

Developer implementation guide: DD_SUB-WF-RELOCATION-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.1.md.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow
DD_SUB-WF-RELOCATION-01-FLOW-WIK-v1.1.md.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.1 | **Satellite of:**

DD_SUB-WF-RELOCATION-01-Subscription_Relocation-v1.1 KE WIK reference BPMN for subscription relocation. Process key: sub-relocate-wik. Composes 11 workers, 3 message catches (cancel + payment + WO-finalized), 1 Camunda intermediate timer (scheduled wait), 1 Drools package. Default effectiveTiming = SCHEDULED_AT; default disconnectMode = TECH_DISPATCH for KEEP_EXISTING (forced for NEW_INSTALL). **What changed from v1.0:** - **Replaced two-WO pattern with single SHIFTING WO** per DD_WO-01-CONTRACT-FOR-SUB-WF. Workers wo-create-disconnect + wo-create-install collapsed into a single wo-create-shifting. Message catches disconnection-completed + installation-completed collapsed into workOrderFinalized. - **Removed equipment unbind/bind workers** from the workflow — WO-01 SHIFTING flow handles equipment internally; workflow reads finalEquipmentBindings from workOrderFinalized event payload. - **Workers reduced from 17 → 11** (cleaner workflow; equipment + WO state-management moved to WO-01's domain). - **Real framework events** — workOrderFinalized, workOrderCancelled, workOrderTimedOut. No more fabricated event names.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-RELOCATION-01-FLOW-WIK-v1.1.md
Version	v1.1
DD type	operator BPMN flow
BPMN/process keys	sub-relocate-wik
Drools packages	rules.subscription.relocation.wik

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /subscriptions/sub_01HZ_KAMAU_FIBER/pending-relocation/subop_01HZ_KAMAU_RELOCATE

- POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/relocate
- POST /api/work-orders

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- disconnection-completed
- drools-decide
- ful-call-activate
- ful-call-claim-homepass
- ful-call-release-homepass
- ful-call-resume
- ful-call-suspend
- fulfillment-completed
- fulfillment-failed
- installation-completed
- kafka-emit
- notification-send
- payment-await
- payment-confirmed
- resolve-disconnect-mode
- resolve-effective-timing
- resolve-equipment-path
- sub-lm-commit-state
- sub-lm-put-master-fields
- sub-lm-put-pending-status
- sub-lm-revert-to-prior
- sub-relocate-wik
- subscription-operation-finalize
- subscription-workflow-rules
- validate-preconditions
- validate-target-homepass
- wo-create-disconnect
- wo-create-install
- wo-create-shifting

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- HomePassActiveTerminationPointsChanged
- SubscriptionRelocated
- SubscriptionRelocationCancelled
- SubscriptionRelocationRejected
- SubscriptionRelocationScheduled
- WorkOrderCancelled
- WorkOrderFinalized
- WorkOrderTimedOut

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.

Concern	Recommendation
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.1 | **Satellite of:** DD_SUB-WF-RELOCATION-01-Subscription_Relocation-v1.1

KE WIK reference BPMN for subscription relocation. Process key: sub-relocate-wik. Composes 11 workers, 3 message catches (cancel + payment + WO-finalized), 1 Camunda intermediate timer (scheduled wait), 1 Drools package. Default `effectiveTiming = SCHEDULED_AT`; default `disconnectMode = TECH_DISPATCH` for `KEEP_EXISTING` (forced for `NEW_INSTALL`).

What changed from v1.0:

- **Replaced two-WO pattern with single SHIFTING WO** per DD_WO-01-CONTRACT-FOR-SUB-WF. Workers `wo-create-disconnect` + `wo-create-install` collapsed into a single `wo-create-shifting`. Message catches `disconnection-completed` + `installation-completed` collapsed into `WorkOrderFinalized`.
- **Removed equipment unbind/bind workers** from the workflow — WO-01 SHIFTING flow handles equipment internally; workflow reads `finalEquipmentBindings` from `WorkOrderFinalized` event payload.
- **Workers reduced from 17 → 11** (cleaner workflow; equipment + WO state-management moved to WO-01's domain).
- **Real framework events** — `WorkOrderFinalized`, `WorkOrderCancelled`, `WorkOrderTimedOut`. No more fabricated event names.

1. What this flow is

KE WIK's BPMN composes framework workers to relocate a subscription from source to target HomePass while keeping the Package unchanged. Trigger endpoint accepts the request, persists `subscription_operation` row, starts process instance of `sub-relocate-wik`. The BPMN orchestrates: Drools validation (preconditions + target-HomePass capability for current Package + equipment-path inference + disconnect-mode resolution + effective-timing) → if non-IMMEDIATE, emit Scheduled event + park on Camunda timer → at fire (or immediately for IMMEDIATE) re-validate → pending state flip → billing intent → pay-first gate if `totals.net > 0` → FUL-04 SUSPEND_SERVICE at source → FUL-07 RELEASE source HomePass → physical work (SHIFTING WO via WO-01, or self-move acknowledgement) → FUL-03 ACTIVATE_SERVICE at target → FUL-07 CLAIM target HomePass → master fields update → commit state ACTIVE → emit event → notify customer → finalize.

Workers composed (11)

#	Worker topic	Used for
1	drools-decide	Validation + resolution: preconditions, target-HomePass capability, equipment-path inference, disconnect-mode, effective-timing
2	sub-lm-put-pending-status	Flip subscription to <code>PENDING_RELOCATION</code> (at commit window)
3	bil01-emit-intent	<code>RELOCATION_INTENT</code> — relocation fee per <code>BIL-CFG-01</code>
4	payment-await	Pay-first gate when <code>totals.net > 0</code>
5	ful-call-suspend	FUL-04 SUSPEND_SERVICE at source HomePass
6	ful-call-release-homepass	FUL-07 RELEASE on source HomePass (decrement ATP, release port bindings, close binding row)
7	wo-create-shifting	Create SHIFTING WO via WO-01 (conditional — <code>disconnectMode = TECH_DISPATCH</code> only). Single worker for the entire physical operation.
8	ful-call-activate	FUL-03 ACTIVATE_SERVICE at target HomePass with <code>finalEquipmentBindings</code> from WO event (or pre-existing for self-move)

#	Worker topic	Used for
9	ful-call-claim-homepass	FUL-07 CLAIM on target HomePass (increment ATP, capture port bindings, open binding row)
10	sub-lm-put-master-fields	Write homepass_id, previous_homepass_id, last_relocated_at, last_status_changed_at
11	sub-lm-commit-state	Flip subscription back to ACTIVE

Plus framework-standard kafka-emit, notification-send, subscription-operation-finalize workers — invoked at acceptance, commit, and finalize per framework patterns.

Message catches + timers

Type	Name / time expression	Used when
Intermediate timer	timeDate=\${scheduledCommitAt}	Non-IMMEDIATE wait
Boundary message on timer	external_cancel_requested (correlated on operationId)	Cancellation during scheduled wait
Intermediate message catch	payment-confirmed (correlated on operationId)	Pay-first gate
Boundary timer on payment catch	timeDuration=P7D	Payment timeout → cancel relocation
Intermediate message catch	WorkOrderFinalized (correlated on callerContext.operationId)	After wo-create-shifting; BPMN waits for SHIFTING WO completion. Skipped when disconnectMode = CUSTOMER_SELF_MOVE.
Boundary message on WO catch	WorkOrderCancelled (correlated on callerContext.operationId)	Dispatcher force-cancels the WO → workflow runs compensation
Boundary message on WO catch	WorkOrderTimedOut (correlated on callerContext.operationId)	WO exceeds wo_kind_config(kind=SHIFTING).default_timeout_days
Intermediate message catch	fulfillment-completed / fulfillment-failed (correlated on fulfillmentCorrelationId)	After each FUL-03/FUL-04/FUL-07 call

Drools package

rules.subscription.relocation.wik — declared decisions:

Decision	Output	Used at
validate-preconditions	eligible: bool, eligibilityReasonCode?, sourceHomepassSnapshot: object, currentPackageSnapshot: object	Beginning — subscription ACTIVE, no in-flight or pending-relocation already, role-trigger match
validate-target-homepass	targetHomepassValid: bool, targetHomepassReasonCode?, targetHomepassSnapshot: object, missingServiceManagementKeys: array, unprovisionedPorts: array, sameFranchise: bool	Target HomePass sellable + supports current Package's required service families + ports resolved + status active
resolve-equipment-path	resolvedEquipmentPath: str, pathReasonCode?	Auto-infer from source/target network-tech compatibility; validate request override
resolve-disconnect-mode	resolvedDisconnectMode: str, disconnectModeReasonCode?	NEW_INSTALL forces TECH_DISPATCH; KEEP_EXISTING uses request or config default
resolve-effective-timing	resolvedEffectiveTiming: str, scheduledCommitAt: timestamp?, timingValid: bool, timingReasonCode?	Resolves request or operator-config default; validates scheduledAt bounds

2. BPMN structure

Process key: sub-relocate-wik. Deployed as Subscription_Relocation_WIK.bpmn.

2.1 Outline

START (event)

■ Process variables from trigger endpoint:

- operationId, subscriptionId, customerId, operatorCode, sourceHomepassId,
- currentPackageRef, relocationTrigger, targetHomepassId,

```

■ equipmentPath?, disconnectMode?, effectiveTiming?, scheduledAt?,
■ notes, initiatingActorUserId, initiatingActorRole, initiatingWorkflowKind,
■ initiatingWorkflowRef, idempotencyKey
■
▼
[STATE: VALIDATING_REQUEST]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.relocation.wik",
         factsJson={subscription state, current Package snapshot, source HomePass snapshot,
         target HomePass snapshot (service_management_endpoints, status_code,
         network_path root node type, franchise),
         equipmentPath request, disconnectMode request,
         effectiveTiming + scheduledAt request,
         role, operator config},
         requestedDecisions=["validate-preconditions", "validate-target-homepass",
         "resolve-equipment-path", "resolve-disconnect-mode",
         "resolve-effective-timing"]
  outputs: <all 5 decisions' outputs>
■
▼ XOR: all checks pass?
■ NO → REJECT subflow (emit SubscriptionRelocationRejected + finalize FAILED)
■ YES ↓
■
▼ XOR: resolvedEffectiveTiming == "IMMEDIATE"?
■ YES ↓
■ NO ↓
■ [STATE: SCHEDULED]
■ Service Task: kafka-emit
■   inputs: topic="sophix.subscription.subscription.relocationscheduled",
■          payloadJson=<SubscriptionRelocationScheduled per parent $6>
■
■ ▼
■ Service Task: notification-send (reliability-isolated)
■   inputs: notificationEventKind="SUBSCRIPTION_RELOCATION_SCHEDULED",
■          eventContext={sourceHomepassId, targetHomepassId, scheduledCommitAt,
■                      resolvedEquipmentPath, resolvedDisconnectMode, sameFranchise}
■
■ ▼
■ Intermediate Catch Event: timer
■   timeDate: ${scheduledCommitAt}
■   Boundary listener: external_cancel_requested
■   → on cancel: emit SubscriptionRelocationCancelled + finalize CANCELLED
■   ▼ timer fires
■
▼
[STATE: VALIDATING (commit-time revalidation)]
Service Task: drools-decide (revalidation)
  inputs: <re-loaded current snapshots>,
         requestedDecisions=["validate-preconditions", "validate-target-homepass",
         "resolve-equipment-path", "resolve-disconnect-mode"]
■
▼ XOR: revalidation passes?
■ NO → REJECT (rejectionStage="COMMIT_REVALIDATION")
■ YES ↓
■
[STATE: PENDING_STATE_FLIP]
Service Task: sub-lm-put-pending-status
  inputs: subscriptionId, toStatus="PENDING_RELOCATION",
         currentTransitionType="RELOCATION",
         currentTransitionReasonCode=<relocationTrigger>
■
▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
         intentCode="RELOCATION_INTENT",
         intentContext={sourceHomepassId, sourceFranchiseId, sourceNetworkRootType,
         targetHomepassId, targetFranchiseId, targetNetworkRootType,
         sameFranchise, equipmentPath=<resolvedEquipmentPath>,
         disconnectMode=<resolvedDisconnectMode>,
         relocationTrigger, scheduledCommitAt?}
  outputs: intentDecisionId, intentDecision (lineItems + totals)
■
▼ XOR: totals.net > 0 AND pay_first_required?
■ NO ↓
■ YES ↓
■ [STATE: AWAITING_PAYMENT]
■ Intermediate Catch Event: payment-confirmed (correlated on operationId)
■ Boundary timer: P7D
■   → on timeout: emit SubscriptionRelocationRejected (PAYMENT_TIMEOUT)
■                   + revert state to ACTIVE + finalize FAILED
■
▼
[STATE: SUSPENDING_SOURCE]

```

```

Service Task: ful-call-suspend
  inputs: subscriptionId, homepassId=<sourceHomepassId>, customerId,
         intent="SUSPEND_SERVICE",
         reasonCategory="SOURCE_DECOMMISSION",
         currentPackageCompositionSnapshot,
         sourceServiceManagementEndpoints,
         fulfillmentCorrelationId=<operationId>:suspend-source
  outputs: suspendFulfillmentId, suspendTerminalState
  ■
  ▼ Message catch: fulfillment-completed | fulfillment-failed
  ■ FAILED → REVERT subflow
  ■ COMPLETED ↓
  ■
[STATE: RELEASING_SOURCE_HOMEPASS]
Service Task: ful-call-release-homepass
  inputs: subscriptionId, homepassId=<sourceHomepassId>, customerId,
         releaseReason="RELOCATION",
         releasePortBinding=<if AT_INSTALLATION then true else false>,
         fulfillmentCorrelationId=<operationId>:release-source
  outputs: releaseFulfillmentId
  ■
  ▼ Message catch: fulfillment-completed | fulfillment-failed
  ■ FAILED → REVERT subflow
  ■ COMPLETED ↓
  ■
  ▼ XOR: resolvedDisconnectMode == "TECH_DISPATCH"?
  ■ NO (CUSTOMER_SELF_MOVE) ↓
  ■   Log self-move acknowledgement step (audit only; no field intervention)
  ■   Variable finalEquipmentBindings := <currentEquipmentBindings>
  ■ YES ↓
  ■   [STATE: AWAITING_WO_FINALIZED]
  ■   Service Task: wo-create-shifting
  ■     inputs: subscriptionId, customerId, operatorCode,
  ■            kind="SHIFTING",
  ■            jobTypeCode=<resolved by WO-01 Drools from sourceNetworkRootType + equipmentPath>,
  ■            homepassRefs={primary: <targetHomepassId>, secondary: <sourceHomepassId>},
  ■            currentEquipmentBindings,
  ■            scheduledAt=<NOW for IMMEDIATE; or original scheduledAt for SCHEDULED_AT>,
  ■            flowContext={equipmentPath: <resolvedEquipmentPath>,
  ■                        disconnectMode: <resolvedDisconnectMode>,
  ■                        sourceHomepassId, targetHomepassId},
  ■            callerContext={subscriptionId, operationId,
  ■                          callerWorkflowKind: "SUB_WF_RELOCATION",
  ■                          callerWorkflowRefId: operationId}
  ■     outputs: shiftingWoId
  ■   ▼
  ■   Intermediate Catch Event: WorkOrderFinalized
  ■     Correlation: callerContext.operationId
  ■     Payload includes: finalEquipmentBindings, accountSubStatusChange, woFinalizedAt
  ■     Boundary message: WorkOrderCancelled (correlated on callerContext.operationId)
  ■       → emit SubscriptionRelocationRejected (WO_CANCELLED_BY_DISPATCHER)
  ■         + REVERT (FUL-07 CLAIM back on source, FUL-04 RESUME at source – best-effort)
  ■         + finalize FAILED
  ■     Boundary message: WorkOrderTimedOut (correlated on callerContext.operationId)
  ■       → emit SubscriptionRelocationRejected (WO_TIMEOUT)
  ■         + REVERT (best-effort) + finalize FAILED
  ■   ▼
[STATE: ACTIVATING_TARGET]
Service Task: ful-call-activate
  inputs: subscriptionId, homepassId=<targetHomepassId>, customerId,
         intent="ACTIVATE_SERVICE",
         currentPackageCompositionSnapshot,
         targetServiceManagementEndpoints,
         equipmentBindings=<finalEquipmentBindings>,
         fulfillmentCorrelationId=<operationId>:activate-target
  outputs: activateFulfillmentId, activateTerminalState
  ■
  ▼ Message catch: fulfillment-completed | fulfillment-failed
  ■ FAILED → REVERT subflow (FUL-04 RESUME at source – best-effort,
  ■                        FUL-07 CLAIM back on source – best-effort,
  ■                        revert state, emit Rejected, finalize FAILED)
  ■ COMPLETED ↓
  ■
[STATE: CLAIMING_TARGET_HOMEPASS]
Service Task: ful-call-claim-homepass
  inputs: subscriptionId, homepassId=<targetHomepassId>, customerId,
         claimReason="RELOCATION",
         portBindings=<from finalEquipmentBindings if AT_INSTALLATION>,
         fulfillmentCorrelationId=<operationId>:claim-target
  outputs: claimFulfillmentId
  ■

```

```

▼ Message catch: fulfillment-completed | fulfillment-failed
■ FAILED → manual recovery (target activated but binding not opened – operational alert)
■ COMPLETED ↓
■
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-master-fields
  inputs: subscriptionId,
         fieldsJson={
           homepass_id: <targetHomepassId>,
           previous_homepass_id: <sourceHomepassId>,
           last_relocated_at: NOW,
           last_status_changed_at: NOW
         }
■
▼
Service Task: sub-lm-commit-state
  inputs: subscriptionId, toStatus="ACTIVE", expectedFromStatus="PENDING_RELOCATION"
■
▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
  inputs: topic="sophix.subscription.subscription.relocated",
         key=<subscriptionId>,
         payloadJson=<full SubscriptionRelocated payload per parent §6>
■
▼
Service Task: notification-send (reliability-isolated, conditional)
  inputs: recipientKind="customer", recipientRef=<customerId>,
         notificationEventKind="SUBSCRIPTION_RELOCATED",
         eventContext={sourceHomepassId, targetHomepassId, sameFranchise,
                       resolvedEquipmentPath, resolvedDisconnectMode,
                       shiftingWoId, finalEquipmentBindings,
                       actualCommitAt, relocationTrigger}
■ failure here does NOT fail the BPMN
▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
  inputs: operationId, finalState="COMPLETED"
■
END (event)

```

2.2 BPMN XML excerpts

Scheduled wait timer with cancel boundary:

```

<bpmn:intermediateCatchEvent id="Catch_scheduled_timer">
  <bpmn:timerEventDefinition>
    <bpmn:timeDate>${scheduledCommitAt}</bpmn:timeDate>
  </bpmn:timerEventDefinition>
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="Boundary_external_cancel" attachedToRef="Catch_scheduled_timer" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_external_cancel_requested" />
</bpmn:boundaryEvent>

```

SHIFTING WO message catch with cancel + timeout boundaries:

```

<bpmn:intermediateCatchEvent id="Catch_wo_finalized">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderFinalized" />
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="Boundary_wo_cancelled" attachedToRef="Catch_wo_finalized" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderCancelled" />
</bpmn:boundaryEvent>

<bpmn:boundaryEvent id="Boundary_wo_timedout" attachedToRef="Catch_wo_finalized" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderTimedOut" />
</bpmn:boundaryEvent>

```

Message correlation: `${callerContext.operationId}` extracted from the event payload matches the BPMN process variable `operationId`.

2.3 Compensation paths

If this step fails	Compensation runs
drools-decide validation (request OR commit revalidation)	No mutations yet; emit Rejected; finalize FAILED
Scheduled wait — external cancel	Emit Cancelled; finalize CANCELLED
sub-lm-put-pending-status	Atomic; failure leaves subscription ACTIVE on source; emit Rejected; finalize FAILED

If this step fails	Compensation runs
bil01-emit-intent	sub-lm-revert-to-prior to ACTIVE; emit Rejected; finalize FAILED
Pay-first timeout (7 days)	sub-lm-revert-to-prior; emit Rejected (PAYMENT_TIMEOUT); finalize FAILED
ful-call-suspend at source fails	sub-lm-revert-to-prior; emit Rejected; finalize FAILED. Source service may be partially down — manual ops alert.
ful-call-release-homepass at source fails	ful-call-resume at source (best-effort); sub-lm-revert-to-prior; emit Rejected; finalize FAILED
wo-create-shifting fails	FUL-07 CLAIM back on source; FUL-04 RESUME at source (best-effort); sub-lm-revert-to-prior; emit Rejected; finalize FAILED
WorkOrderCancelled (dispatcher cancelled WO)	FUL-07 CLAIM back on source (best-effort); FUL-04 RESUME at source (best-effort); sub-lm-revert-to-prior; emit Rejected (WO_CANCELLED_BY_DISPATCHER); finalize FAILED
WorkOrderTimedOut	FUL-07 CLAIM back on source (best-effort); FUL-04 RESUME at source (best-effort); sub-lm-revert-to-prior; emit Rejected (WO_TIMEOUT); finalize FAILED
ful-call-activate at target fails	FUL-07 CLAIM back on source (best-effort); FUL-04 RESUME at source (best-effort); revert state; emit Rejected. WO already finalized — equipment installed at target without service activation — operational alert.
ful-call-claim-homepass at target fails	Manual recovery — target service running but FUL-07 binding not open; FUL-07 emits operational alert.
sub-lm-put-master-fields	Manual recovery — service running at target but master fields not updated
sub-lm-commit-state	Manual recovery
kafka-emit	Retried by outbox dispatcher; BPMN proceeds
notification-send	Logged but NOT compensated (reliability-isolated)

2.4 Configuration row (KE WIK)

In subscription_operation_config (framework parent):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
WIK	RELOCATION	sub-relocate-wik	5184000	{}

(Timeout: 60 days — accommodates max_future_scheduled_days for SCHEDULED_AT case.)

In subscription_relocation_config (parent §3.1):

operator_code	customer_self_service_enabled	pay_first_required	default_effective_timing	max_future_scheduled_days	default_disconnect_mode_keep_existing	customer_notification_enabled
WIK	true	true	SCHEDULED_AT	60	TECH_DISPATCH	true

In WO-01's wo_kind_config (referenced; not owned by this DD):

operator_code	kind	default_timeout_days
WIK	SHIFTING	14

(The single WO-01 timeout governs the entire SHIFTING WO including both disconnect and reconnect phases.)

3. Drools package rules.subscription.relocation.wik

KE WIK rules deployed as part of subscription-workflow-rules KJAR. Decision shapes mirror prior workflow DDs with relocation-specific adaptations.

3.1 validate-preconditions

```
package rules.subscription.relocation.wik

import com.sophix.subwf.facts.RelocationRequestFact
import com.sophix.subwf.facts.SubscriptionStateFact
import com.sophix.subwf.facts.OperatorConfigFact

rule "Eligibility: subscription must be ACTIVE"
when
```

```

    $req: RelocationRequestFact()
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
  then
    modify($req) { setEligible(false); setEligibilityReasonCode("INVALID_STATE_FOR_RELOCATION") }
  end

rule "Eligibility: no in-flight operation"
when
  $req: RelocationRequestFact(eligible == null)
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, currentTransitionType != null)
then
  modify($req) { setEligible(false); setEligibilityReasonCode("OPERATION_IN_PROGRESS") }
end

rule "Eligibility: CUSTOMER_REQUESTED_RELOCATION requires customer / CC / operator role"
when
  $req: RelocationRequestFact(relocationTrigger == "CUSTOMER_REQUESTED_RELOCATION", eligible == null,
    initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR"))
then
  modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: ADMIN_RELOCATION requires SUBSCRIPTION_ADMIN or SUBSCRIPTION_OPERATOR"
when
  $req: RelocationRequestFact(relocationTrigger == "ADMIN_RELOCATION", eligible == null,
    initiatingActorRole not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR"))
then
  modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: default-pass"
salience -100
when
  $req: RelocationRequestFact(eligible == null)
then
  modify($req) { setEligible(true) }
end

```

3.2 validate-target-homepass

```

import com.sophix.subwf.facts.HomePassFact
import com.sophix.subwf.facts.PackageFact

rule "Target HomePass: not sellable"
when
  $req: RelocationRequestFact(eligible == true, targetHomepassValid == null)
  $tgt: HomePassFact(id == $req.targetHomepassId, isSellable == false)
then
  modify($req) {
    setTargetHomepassValid(false);
    setTargetHomepassReasonCode("TARGET_HOMEPASS_NOT_SELLABLE")
  }
end

rule "Target HomePass: same as source"
when
  $req: RelocationRequestFact(eligible == true, targetHomepassValid == null,
    targetHomepassId == sourceHomepassId)
then
  modify($req) {
    setTargetHomepassValid(false);
    setTargetHomepassReasonCode("SAME_HOMEPASS")
  }
end

rule "Target HomePass: missing current Package's required service-management keys"
when
  $req: RelocationRequestFact(eligible == true, targetHomepassValid == null)
  $tgt: HomePassFact(id == $req.targetHomepassId)
  $pkg: PackageFact(packageRef == $req.currentPackageRef)
  eval(!$tgt.getServiceManagementEndpointsKeys().containsAll($pkg.getRequiredServiceManagementKeys()))
then
  java.util.Set missing = new java.util.HashSet($pkg.getRequiredServiceManagementKeys());
  missing.removeAll($tgt.getServiceManagementEndpointsKeys());
  modify($req) {
    setTargetHomepassValid(false);
    setTargetHomepassReasonCode("CURRENT_PACKAGE_NOT_AVAILABLE_AT_TARGET_HOMEPASS");
    setMissingServiceManagementKeys(new java.util.ArrayList(missing))
  }
end

rule "Target HomePass: ports unprovisioned for IMMEDIATE"

```

```

when
  $req: RelocationRequestFact(eligible == true, targetHomepassValid == null,
    effectiveTiming == "IMMEDIATE")
  $tgt: HomePassFact(id == $req.targetHomepassId)
  $pkg: PackageFact(packageRef == $req.currentPackageRef)
  eval($tgt.hasNullPortsForKeys($pkg.getRequiredServiceManagementKeys()))
then
  modify($req) {
    setTargetHomepassValid(false);
    setTargetHomepassReasonCode("TARGET_HOMEPASS_PORT_NOT_PROVISIONED")
  }
end

rule "Target HomePass: default-pass + capture context"
salience -100
when
  $req: RelocationRequestFact(eligible == true, targetHomepassValid == null)
  $tgt: HomePassFact(id == $req.targetHomepassId)
  $src: HomePassFact(id == $req.sourceHomepassId)
then
  modify($req) {
    setTargetHomepassValid(true);
    setTargetHomepassSnapshot($tgt);
    setSourceHomepassSnapshot($src);
    setSameFranchise($tgt.getFranchiseId().equals($src.getFranchiseId()))
  }
end

```

3.3 resolve-equipment-path

```

rule "Equipment path: cross-tech forces NEW_INSTALL"
when
  $req: RelocationRequestFact(targetHomepassValid == true, resolvedEquipmentPath == null)
  eval(!$req.getSourceNetworkRootType().equals($req.getTargetNetworkRootType()))
then
  modify($req) { setResolvedEquipmentPath("NEW_INSTALL") }
end

rule "Equipment path: request KEEP_EXISTING with cross-tech is rejected"
when
  $req: RelocationRequestFact(targetHomepassValid == true,
    equipmentPath == "KEEP_EXISTING",
    resolvedEquipmentPath == null)
  eval(!$req.getSourceNetworkRootType().equals($req.getTargetNetworkRootType()))
then
  modify($req) {
    setPathReasonCode("EQUIPMENT_INCOMPATIBLE_WITH_TARGET");
    setTargetHomepassValid(false)
  }
end

rule "Equipment path: same-tech + request omits → KEEP_EXISTING"
when
  $req: RelocationRequestFact(targetHomepassValid == true,
    equipmentPath == null,
    resolvedEquipmentPath == null)
  eval($req.getSourceNetworkRootType().equals($req.getTargetNetworkRootType()))
then
  modify($req) { setResolvedEquipmentPath("KEEP_EXISTING") }
end

rule "Equipment path: request NEW_INSTALL always accepted"
when
  $req: RelocationRequestFact(targetHomepassValid == true,
    equipmentPath == "NEW_INSTALL",
    resolvedEquipmentPath == null)
then
  modify($req) { setResolvedEquipmentPath("NEW_INSTALL") }
end

rule "Equipment path: request KEEP_EXISTING + same-tech → KEEP_EXISTING"
when
  $req: RelocationRequestFact(targetHomepassValid == true,
    equipmentPath == "KEEP_EXISTING",
    resolvedEquipmentPath == null)
  eval($req.getSourceNetworkRootType().equals($req.getTargetNetworkRootType()))
then
  modify($req) { setResolvedEquipmentPath("KEEP_EXISTING") }
end

```

3.4 resolve-disconnect-mode

```

rule "Disconnect mode: NEW_INSTALL forces TECH_DISPATCH"
when
    $req: RelocationRequestFact(resolvedEquipmentPath == "NEW_INSTALL",
                                resolvedDisconnectMode == null)
    eval($req.getDisconnectMode() == null || "TECH_DISPATCH".equals($req.getDisconnectMode()))
then
    modify($req) { setResolvedDisconnectMode("TECH_DISPATCH") }
end

rule "Disconnect mode: NEW_INSTALL + CUSTOMER_SELF_MOVE request is rejected"
when
    $req: RelocationRequestFact(resolvedEquipmentPath == "NEW_INSTALL",
                                disconnectMode == "CUSTOMER_SELF_MOVE",
                                resolvedDisconnectMode == null)
then
    modify($req) {
        setDisconnectModeReasonCode("DISCONNECT_MODE_INCOMPATIBLE_WITH_EQUIPMENT_PATH");
        setTargetHomepassValid(false)
    }
end

rule "Disconnect mode: KEEP_EXISTING – use request value if supplied"
when
    $req: RelocationRequestFact(resolvedEquipmentPath == "KEEP_EXISTING",
                                disconnectMode != null,
                                resolvedDisconnectMode == null)
    eval("TECH_DISPATCH".equals($req.getDisconnectMode()) || "CUSTOMER_SELF_MOVE".equals($req.getDisconnectMode()))
then
    modify($req) { setResolvedDisconnectMode($req.getDisconnectMode()) }
end

rule "Disconnect mode: KEEP_EXISTING – default from operator config"
when
    $req: RelocationRequestFact(resolvedEquipmentPath == "KEEP_EXISTING",
                                disconnectMode == null,
                                resolvedDisconnectMode == null)
    $cfg: OperatorConfigFact()
then
    modify($req) { setResolvedDisconnectMode($cfg.getDefaultDisconnectModeKeepExisting()) }
end

```

3.5 resolve-effective-timing

Standard pattern: resolves request or operator-config default; validates scheduledAt bounds. Same shape as RELOCATION-01 parent §4.8.

4. Worked example — Kamau's scheduled relocation Karen → Lavington

Context: Kamau is on pkg_INET_100M_VOIP_RES at HomePass hp_01HZ_KAREN_017 (same-tech with target). He's moving to Lavington on June 1 — same Package, different HomePass. Calls CC on May 15 to schedule.

May 15, 14:00:00 — Trigger endpoint:

```

POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/relocate
Idempotency-Key: relocate-sub_01HZ_KAMAU-hp_LAVI_073-20260601

{ "operatorCode": "WIK", "relocationTrigger": "CUSTOMER_REQUESTED_RELOCATION",
  "targetHomepassId": "hp_01HZ_LAVI_073",
  "effectiveTiming": "SCHEDULED_AT",
  "scheduledAt": "2026-06-01T09:00:00+03:00",
  "notes": "Customer moving to new apartment in Lavington",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER" }

→ 202 Accepted
{ "operationId": "subop_01HZ_KAMAU_RELOCATE",
  "resolvedEquipmentPath": "KEEP_EXISTING",
  "resolvedDisconnectMode": "TECH_DISPATCH",
  "resolvedEffectiveTiming": "SCHEDULED_AT",
  "scheduledCommitAt": "2026-06-01T09:00:00+03:00",
  "currentState": "SCHEDULED" }

```

May 15, 14:00:00.5 — VALIDATING_REQUEST: Drools: subscription ACTIVE ✓, no in-flight ops ✓, role CALL_CENTER on CUSTOMER_REQUESTED_RELOCATION ✓. Target HomePass status RFS ✓, service_management_endpoints keys cover current Package's required keys ✓, ports non-null ✓. Same network root type → resolvedEquipmentPath =

KEEP_EXISTING. disconnectMode omitted → default TECH_DISPATCH. Effective timing SCHEDULED_AT, within 60-day window ✓.

May 15, 14:00:01 — SCHEDULED: kafka-emit publishes SubscriptionRelocationScheduled. NOT-01 confirms to Kamau: "Your move to Lavington is scheduled for June 1." BPMN parks on Camunda timer. Kamau stays ACTIVE on Karen.

June 1, 09:00:00 — Timer fires. BPMN advances to VALIDATING (commit-time revalidation). All checks re-pass.

sub-lm-put-pending-status → PENDING_RELOCATION.

June 1, 09:00:02 — BILLING_CALL: bil01-emit-intent calls BIL-01 with RELOCATION_INTENT. BIL-CFG-01 has RELOCATION_FEE_CROSS_FRANCHISE matching (sameFranchise=false); returns
lineItems=[{code:"RELOCATION_FEE_CROSS_FRANCHISE", amount:500.00}], totals.net=500.00.

June 1, 09:00:03 — AWAITING_PAYMENT: Pay-first gate. CC agent processes M-Pesa payment. payment-confirmed at 09:00:42.

June 1, 09:00:44 — SUSPENDING_SOURCE: ful-call-suspend → FUL-04 with intent=SUSPEND_SERVICE, reasonCategory=SOURCE_DECOMMISSION at source HomePass. Source service deactivated at 09:00:46.

June 1, 09:00:47 — RELEASING_SOURCE_HOMEPASS: ful-call-release-homepass → FUL-07 RELEASE on source. Source ATP decrement, port bindings released, source homepass_subscription_binding row closed. HomePassActiveTerminationPointsChanged emitted. Returns at 09:00:48.

June 1, 09:00:49 — AWAITING_WO_FINALIZED: wo-create-shifting calls WO-01 POST /api/work-orders with kind=SHIFTING, operator-specific jobTypeCode resolved by WO-01 Drools (e.g., a same-tech disconnect-reconnect code), homepassRefs.secondary=hp_01HZ_KAREN_017, homepassRefs.primary=hp_01HZ_LAVI_073, flowContext={equipmentPath:KEEP_EXISTING, disconnectMode:TECH_DISPATCH, ...}, callerContext.operationId=subop_01HZ_KAMAU_RELOCATE. WO wo_01HZ_KAMAU_SHIFT_3201 created. Tech dispatched. WO-01 SHIFTING flow runs internally: DISCONNECT phase (no site visit; system de-provision at source), phase transition to RECONNECT, tech arrives Lavington with existing equipment from Karen pickup, installs at new fiber drop, captures topology, finalizes. Total WO time: ~1.5 hours. WO-01 emits WorkOrderFinalized at 10:32:00 with payload {shiftingWoId, finalEquipmentBindings:[ONT eqp_ONT_42→INET, ATA eqp_ATA_19→VOIP], accountSubStatusChange:{...}, callerContext.operationId:subop_01HZ_KAMAU_RELOCATE, ...}. BPMN catches.

June 1, 10:32:02 — ACTIVATING_TARGET: ful-call-activate → FUL-03 with intent=ACTIVATE_SERVICE at target HomePass, equipment bindings from WO event. FUL-03 dispatches per-service-class provisioning at target OLT + VOIP switch using service_management_endpoints[data] and [voice]. Returns at 10:32:05.

June 1, 10:32:07 — CLAIMING_TARGET_HOMEPASS: ful-call-claim-homepass → FUL-07 CLAIM on target. Target ATP increment, port bindings captured, target homepass_subscription_binding row opened. Returns at 10:32:08.

June 1, 10:32:10 — COMMITTING_FINAL_STATE: sub-lm-put-master-fields writes homepass_id=hp_01HZ_LAVI_073, previous_homepass_id=hp_01HZ_KAREN_017, last_relocated_at=NOW. sub-lm-commit-state flips PENDING_RELOCATION → ACTIVE.

June 1, 10:32:12 — EMITTING_EVENT: kafka-emit publishes SubscriptionRelocated with shiftingWoId, sourceHomepassId, targetHomepassId, sameFranchise:false, ... notification-send → NOT-01 confirms to Kamau.

June 1, 10:32:13 — COMPLETED.

Total wall time June 1: trigger 09:00 → committed 10:32:13. Subscription stayed ACTIVE on Karen until target activation succeeded; PENDING_RELOCATION only between source-suspend (09:00:44) and final-commit (10:32:12).

Alternative path — NEW_INSTALL (cross-tech relocation)

If Kamau's target HomePass were on a different network tech, Drools auto-infers resolvedEquipmentPath=NEW_INSTALL, forces resolvedDisconnectMode=TECH_DISPATCH. The SHIFTING WO uses an operator-specific job-type code for cross-tech moves; the WO module handles old-equipment recovery + new-equipment installation internally. The WorkOrderFinalized payload carries finalEquipmentBindings with new equipment serials captured at install-finalize. Equipment-replacement-fee BillableEvent may match per BIL-CFG-01 filters.

Alternative path — disconnectMode = CUSTOMER_SELF_MOVE

If Kamau requested self-move and resolvedEquipmentPath=KEEP_EXISTING, no SHIFTING WO is created. BPMN logs the self-move acknowledgement at the AWAITING_WO_FINALIZED state and proceeds. finalEquipmentBindings = currentEquipmentBindings. Kamau handles physical disconnection + transport + reconnection himself.

ACTIVATING_TARGET still runs (FUL-03 needs to provision the customer's connection at the target HomePass — adapter pushes the speed profile + voice registration). This path is enabled in KE WIK config but conservative defaults route most customers through TECH_DISPATCH.

Cancellation flow

If on May 25 Kamau called to cancel: DELETE /subscriptions/sub_01HZ_KAMAU_FIBER/pending-relocation/subop_01HZ_KAMAU_RELOCATE with cancellationReasonCode=CUSTOMER_CHANGED_MIND. Cancel REST sends external_cancel_requested; Camunda boundary on the scheduled-timer fires; BPMN emits SubscriptionRelocationCancelled, finalizes CANCELLED. Subscription stays on Karen. No WO created, no fee charged.

WO timeouts and dispatcher cancellation

If after the SHIFTING WO is created, the tech never shows up and the WO exceeds `wo_kind_config(kind=SHIFTING).default_timeout_days` (14 days), WO-01 emits `WorkOrderTimedOut`. BPMN's boundary catch fires: runs compensation (FUL-07 CLAIM back on source, FUL-04 RESUME at source — both best-effort), sub-`lm-revert-to-prior` flips `PENDING_RELOCATION` → `ACTIVE`, emits `SubscriptionRelocationRejected` with `rejectionReasonCode=WO_TIMEOUT`. Operational alert fires if compensation can't fully restore source service.

If the dispatcher force-cancels the WO mid-flow via WO-01's cancel endpoint, `WorkOrderCancelled` fires the BPMN's boundary catch — same compensation chain with `rejectionReasonCode=WO_CANCELLED_BY_DISPATCHER`.

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-RELOCATION-01 (parent) v1.1	This satellite implements parent's contract	Triggers, dual validation, equipment-path + disconnect-mode + effective-timing all defined in parent
SUB-WF-FRAMEWORK-01 + WORKERS	This satellite composes framework primitives	11 workers composed
SUB-LM-01	Workers write status + master fields atomically	<code>homepass_id</code> + <code>previous_homepass_id</code> + <code>last_relocated_at</code>
RLM-CFG-01	Workers read source + target HomePass	<code>service_management_endpoints</code> , <code>status_code</code> , <code>network_path</code> root node type, franchise
SIP-01	Workers read current Package	Service composition
PLM-CFG-01 v1.0	Workers read service definitions	<code>service_management_key</code> for capability check
BIL-01 v1.1	<code>bil01-emit-intent</code>	RELOCATION_INTENT
BIL-CFG-01 v1.0	BillableEvent catalog	Relocation fee + equipment-replacement fee definitions
FUL-04 v1.1	<code>ful-call-suspend</code>	SUSPEND_SERVICE at source
FUL-03 v1.1	<code>ful-call-activate</code>	ACTIVATE_SERVICE at target
FUL-07 v1.0	<code>ful-call-release-homepass</code> + <code>ful-call-claim-homepass</code>	HomePass bookkeeping
WO-01 (FRAMEWORK + FLOW-SHIFTING)	<code>wo-create-shifting</code> worker (conditional on <code>disconnectMode = TECH_DISPATCH</code>)	Per DD_WO-01-CONTRACT-FOR-SUB-WF
NOT-01	<code>notification-send</code>	Customer notification
BIL-02 / CVM / EM-04	Consume Kafka events	Downstream reporting

5.1 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.
- **Operator-specific job-type code mapping** — each operator's WO-01 Drools KJAR resolves (`kind=SHIFTING`, `sourceNetworkRootType`, `equipmentPath`) → operator job-type code.

B — Current TZ

Status: To be filled in after codebase cartography review.